

LAPORAN
WORKSHOP SISTEM INFORMASI WEB SERVER
(TIF330805)
MINGGU 7 PERTEMUAN 1
SEMESTER III



BKPM ACARA 13

Nama:
SEPTIAN DWI RAMADHAN PUTRA

NIM:
E41250282

Golongan:
A

PROGRAM STUDI DILUAR KAMPUS UTAMA (PSDKU)
TEKNIK INFORMATIKA KAB. NGANJUK
JURUSAN TEKNOLOGI INFORMASI
POLITEKNIK NEGERI JEMBER
TAHUN 2026

PENDAHULUAN

1. Latar Belakang

Pada praktikum sebelumnya, proses tambah dan ubah data mahasiswa masih bertumpu pada Controller dan Entity: Controller membuat object, menangkap exception dari setter, lalu mengatur pesan kesalahan. Selama aplikasi masih kecil hal ini masih dapat dikelola. Namun, ketika aturan bertambah (NIM wajib diisi, NIM tidak boleh sama, program studi harus tersedia, dan seterusnya), Controller akan menjadi panjang, sulit dibaca, dan sulit dipelihara.

Untuk mengatasi hal tersebut digunakan Service Layer. Service menjadi tempat utama logika bisnis (validasi dan aturan aplikasi), Controller hanya mengatur alur request dan response, sedangkan Repository hanya menjalankan query ke database. Hasil proses dikomunikasikan kepada pengguna melalui Flash Message berbasis Session, yaitu pesan yang tampil sekali lalu dihapus.

Pada acara ini, database project juga disamakan dengan file acara_7.sql (database acara_7). Struktur tabel mahasiswa pada database tersebut tidak memiliki kolom email, sehingga project hasil Acara 10 disesuaikan terlebih dahulu sebelum Service Layer dibuat.

2. Tujuan

1. Membuat class MahasiswaService yang menerima MahasiswaRepository dan ProdiRepository melalui constructor.
2. Memindahkan validasi data mahasiswa dari Controller ke Service.
3. Membuat method create() dan update() pada Service untuk proses tambah dan ubah data.
4. Menyederhanakan Controller sehingga tidak lagi melakukan validasi dan query database secara langsung.
5. Membuat Flash Message dengan \$_SESSION untuk pesan berhasil ditambahkan, berhasil diubah, gagal disimpan, dan NIM sudah terdaftar, lalu menghapusnya setelah ditampilkan.
6. Menyamakan konfigurasi dan struktur project dengan database acara_7.

3. Dasar Teori Singkat

Service Layer. Lapisan yang menampung logika bisnis aplikasi. Controller tidak perlu mengetahui bagaimana data divalidasi, dan Repository tidak perlu mengetahui aturan bisnis. Dengan pembagian ini kode lebih mudah dibaca, diuji, dan dikembangkan.

Pembagian tugas. Controller mengatur alur (menerima request lalu memilih response), Service mengatur logika bisnis (memeriksa NIM, prodi, dan sebagainya), Repository mengatur database (menjalankan query SELECT, INSERT, UPDATE, DELETE).

Validasi. Pemeriksaan data masukan sebelum disimpan, misalnya wajib diisi, format benar, dan tidak duplikat. Validasi di sisi server wajib ada karena validasi di browser (atribut required) dapat dilewati pengguna.

Session dan Flash Message. Session menyimpan data sementara antar request. Flash Message adalah pesan yang disimpan di session, ditampilkan satu kali pada halaman berikutnya, lalu dihapus. Alurnya mengikuti pola Post/Redirect/Get sehingga pesan tidak muncul lagi saat halaman di-refresh.

4. Alat dan Bahan

- Laptop/PC
- Web Browser
- Visual Studio Code
- XAMPP/Laragon (Apache + MySQL/MariaDB)
- PHP 8.2
- phpMyAdmin dan file acara_7.sql
- HTML, CSS, Bootstrap
- Git
- Modul Praktikum (BKPM Acara 13)

HASIL PRAKTIK

1. Persiapan: Branch dan Penyesuaian Database acara_7

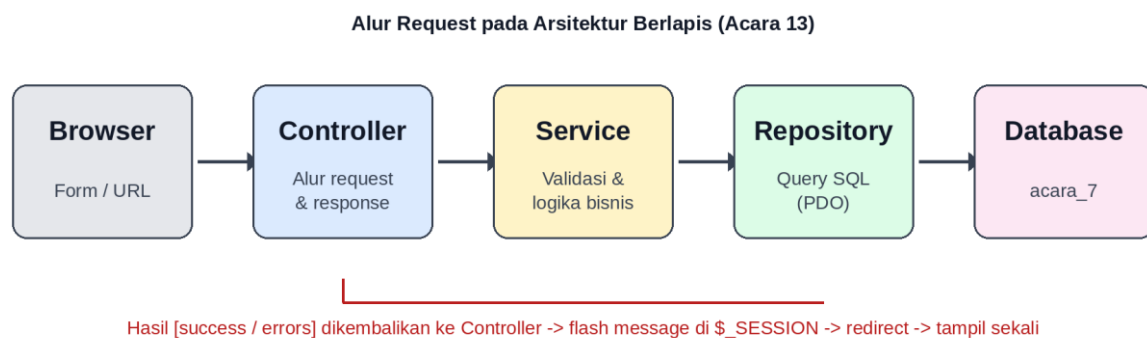
Project yang dipakai adalah si-akademik hasil Acara 10 yang sudah dikelola dengan Git pada Acara 12. Pekerjaan Acara 13 dikerjakan pada branch terpisah bernama fitur-service-layer (sesuai alur kerja Acara 12), lalu digabungkan ke main setelah selesai. Riwayat commit dan ringkasan perubahannya:

```
PS C:\xampp\htdocs\acara13> git checkout fitur-service-layer
Switched to branch 'fitur-service-layer'
PS C:\xampp\htdocs\acara13> git log --oneline fitur-service-layer
5ad4bde (HEAD -> fitur-service-layer) Memperbarui README untuk Acara 13
ee05447 Merombak MahasiswaController agar tipis dan menambahkan flash message sukses/gagal
61aa589 Menambahkan ProdiRepository dan MahasiswaService (validasi dipindah ke Service)
8ea9efe Menyesuaikan project dengan database acara_7 (tanpa kolom email)
f814d16 Memperbarui README untuk Acara 12
b243739 Menyelesaikan merge conflict
64840a0 Menambahkan nama PSDKU pada subjudul login
964ad20 (fitur-judul-login) Mengubah subjudul halaman login
4f41dfa (fitur-login) Menambahkan validasi input kosong dan mempertahankan username saat login gagal
5f0800b Menambahkan tombol lihat/sembunyikan password pada halaman login
4902616 Commit awal project si-akademik (hasil Acara 10)
(END)

PS C:\xampp\htdocs\acara13> git diff --stat f814d16..fitur-service-layer
 README.md | 32 +++--
 acara_7.sql | 167 +++++
 app/Controllers/MahasiswaController.php | 125 +++++-----
 app/Models/Mahasiswa.php | 14 +--
 app/Repositories/MahasiswaRepository.php | 30 +++-
 app/Repositories/ProdiRepository.php | 37 +++++
 app/Services/MahasiswaService.php | 192 +++++
 README.md | 32 +++--
 acara_7.sql | 167 +++++
 app/Controllers/MahasiswaController.php | 125 +++++-----
 app/Models/Mahasiswa.php | 14 +--
 app/Repositories/MahasiswaRepository.php | 30 +++-
 app/Repositories/ProdiRepository.php | 37 +++++
 app/Services/MahasiswaService.php | 192 +++++
 app/Services/README.md | 6 +-
 app/Views/mahasiswa/create.php | 25 +--
 app/Views/mahasiswa/detail.php | 4 -
 app/Views/mahasiswa/edit.php | 40 +++--
 app/Views/mahasiswa/index.php | 9 +-
 app/Views/partials/flash.php | 13 +++
 config/database.php | 7 +-
 public/assets/css/app.css | 7 ++
 public/index.php | 2 +
 si_akademik.sql | 89 -----
17 files changed, 587 insertions(+), 212 deletions(-)
```

2. Arsitektur Controller, Service, dan Repository

Sebelum membuat Service, pembagian tanggung jawab antar lapisan ditentukan seperti pada diagram berikut. Request dari browser diterima Controller, logika bisnis dikerjakan Service, dan akses database dilakukan Repository melalui koneksi PDO singleton.



Gambar 5. Alur request pada arsitektur berlapis

3. ProdiRepository dan Pengecekan NIM pada Repository

Service memerlukan dua kemampuan dari lapisan data: memastikan program studi yang dipilih ada, dan memastikan NIM belum dipakai. Untuk itu dibuat ProdiRepository (method all, find, dan exists) serta ditambahkan method existsByNim() pada MahasiswaRepository. Parameter \$exceptId dipakai saat proses ubah agar NIM milik data itu sendiri tidak dianggap duplikat. Method create() kini juga mengembalikan id data baru. Seluruh query memakai prepared statement.

```
// Mengecek apakah NIM sudah dipakai. $exceptId dipakai saat update agar
// baris milik data itu sendiri tidak ikut dihitung (Acara 13).
public function existsByNim(string $nim, ?int $exceptId = null): bool
{
    $sql = "SELECT COUNT(*) FROM mahasiswa WHERE nim = :nim";
    $params = ['nim' => $nim];

    if ($exceptId !== null) {
        $sql .= " AND id <> :except_id";
        $params['except_id'] = $exceptId;
    }

    $stmt = $this->db->prepare($sql);
    $stmt->execute($params);

    return (int) $stmt->fetchColumn() > 0;
}

public function create(Mahasiswa $mhs): int
{
    $stmt = $this->db->prepare(
        "INSERT INTO mahasiswa (nim, nama, prodi_id, angkatan, status)
        VALUES (:nim, :nama, :prodi_id, :angkatan, :status)"
    );
```

```

    );
    $stmt->execute($this->toParams($mhs));

    return (int) $this->db->lastInsertId();
}

```

Gambar 6. MahasiswaRepository: existsByNim() dan create() yang mengembalikan id

```

<?php
// app/Repositories/ProdiRepository.php
// Repository untuk tabel prodi. Dipakai oleh MahasiswaService (Acara 13,
// langkah 2) untuk memastikan program studi yang dipilih benar-benar ada.
// Koneksi PDO diterima lewat constructor (Constructor Injection).

class ProdiRepository
{
    private PDO $db;

    public function __construct(PDO $db)
    {
        $this->db = $db;
    }

    public function all(): array
    {
        return $this->db->query("SELECT * FROM prodi ORDER BY id")->fetchAll();
    }

    public function find(int $id): ?array
    {
        $stmt = $this->db->prepare("SELECT * FROM prodi WHERE id = :id");
        $stmt->execute(['id' => $id]);
        $row = $stmt->fetch();

        return $row ?: null;
    }

    public function exists(int $id): bool
    {
        $stmt = $this->db->prepare("SELECT COUNT(*) FROM prodi WHERE id = :id");
        $stmt->execute(['id' => $id]);

        return (int) $stmt->fetchColumn() > 0;
    }
}

```

Gambar 7. app/Repositories/ProdiRepository.php

4. MahasiswaService

Class MahasiswaService dibuat pada app/Services/MahasiswaService.php (langkah 1). MahasiswaRepository dan ProdiRepository diterima melalui constructor (langkah 2). Method create() dan update() (langkah 4 dan 5) tidak melempar exception ke Controller, melainkan mengembalikan array hasil dengan kunci success dan errors, sehingga Controller cukup memeriksa hasil tersebut.

```

class MahasiswaService
{
    public const STATUS_VALID = ['aktif', 'cuti', 'lulus'];

    private MahasiswaRepository $repo;
    private ProdiRepository $prodiRepo;

    public function __construct(MahasiswaRepository $repo, ProdiRepository
$prodiRepo)
    {
        $this->repo      = $repo;
        $this->prodiRepo = $prodiRepo;
    }

    // ===== QUERY (diteruskan ke Repository) =====
    public function list(?string $keyword = null): array
    {
        return $this->repo->allWithProdi($keyword);
    }

    public function findById(int $id): ?Mahasiswa
    {
        return $this->repo->find($id);
    }

    public function findByNim(?string $nim): ?Mahasiswa
    {
        return $this->repo->findByNim($nim);
    }

    public function prodiOptions(): array
    {
        return $this->prodiRepo->all();
    }

    // ===== CREATE (langkah 4) =====
    public function create(array $input): array
    {
        $data    = $this->normalize($input);
        $errors = $this->validate($data);

        if (!empty($errors)) {
            return ['success' => false, 'errors' => $errors];
        }

        try {
            $id = $this->repo->create($this->buildEntity($data));
            return ['success' => true, 'errors' => [], 'id' => $id];
        } catch (InvalidArgumentException | PDOException $e) {
            return ['success' => false, 'errors' => $this->errorFromException($e)];
        }
    }

    // ===== UPDATE (langkah 5) =====
    public function update(int $id, array $input): array
    {
        if ($id <= 0 || $this->repo->find($id) === null) {
            return [
                'success'    => false,
                'not_found' => true,
                'errors'     => ['general' => 'Data mahasiswa tidak ditemukan.'],
            ];
        }
    }
}

```

```

$data = $this->normalize($input);
$errors = $this->validate($data, $id);

if (!empty($errors)) {
    return ['success' => false, 'errors' => $errors];
}

try {
    $this->repo->update($id, $this->buildEntity($data));
    return ['success' => true, 'errors' => []];
} catch (InvalidArgumentException | PDOException $e) {
    return ['success' => false, 'errors' => $this->errorFromException($e)];
}
}

```

Gambar 8. MahasiswaService: constructor, create(), dan update()

Seluruh aturan validasi dipindahkan ke method private validate() (langkah 3). Aturan yang diterapkan: NIM wajib, harus angka, maksimal 20 karakter, dan tidak boleh sudah terdaftar; nama wajib dan maksimal 100 karakter; program studi wajib dipilih dan harus ada di database; angkatan antara 2000 dan 2100; status hanya aktif, cuti, atau lulus.

```

private function validate(array $data, ?int $exceptId = null): array
{
    $errors = [];

    // NIM: wajib, angka, maksimal 20 karakter, dan tidak boleh sama.
    if ($data['nim'] === '') {
        $errors['nim'] = 'NIM wajib diisi.';
    } elseif (!ctype_digit($data['nim'])) {
        $errors['nim'] = 'NIM harus berupa angka.';
    } elseif (strlen($data['nim']) > 20) {
        $errors['nim'] = 'NIM maksimal 20 karakter.';
    } elseif ($this->repo->existsByNim($data['nim'], $exceptId)) {
        $errors['nim'] = 'NIM sudah terdaftar.';
    }

    // Nama: wajib, maksimal 100 karakter.
    if ($data['nama'] === '') {
        $errors['nama'] = 'Nama wajib diisi.';
    } elseif (mb_strlen($data['nama']) > 100) {
        $errors['nama'] = 'Nama maksimal 100 karakter.';
    }

    // Program studi: wajib dipilih dan harus ada di database.
    if ($data['prodi_id'] <= 0) {
        $errors['prodi_id'] = 'Program studi wajib dipilih.';
    } elseif (!$this->prodiRepo->exists($data['prodi_id'])) {
        $errors['prodi_id'] = 'Program studi tidak ditemukan.';
    }

    // Angkatan: tahun 2000 sampai 2100.
    if ($data['angkatan'] < 2000 || $data['angkatan'] > 2100) {
        $errors['angkatan'] = 'Angkatan harus berupa tahun antara 2000 dan
2100.';
    }

    // Status: hanya aktif, cuti, atau lulus.
    if (!in_array($data['status'], self::STATUS_VALID, true)) {

```

```

        $errors['status'] = 'Status tidak valid.';
    }

    return $errors;
}

```

Gambar 9. Method validate() pada MahasiswaService

Sebagai lapisan pengaman terakhir, Entity Mahasiswa tetap mempertahankan validasi pada setter (hasil Acara 9), dan Service menangkap `InvalidArgumentException` maupun `PDOException`. Kode kesalahan MySQL 1062 (duplicate entry) diterjemahkan menjadi pesan "NIM sudah terdaftar." untuk kasus dua request bersamaan yang lolos dari pengecekan `existsByNim()`.

5. Controller yang Disederhanakan

MahasiswaController dirombak (langkah 6). Controller kini hanya memiliki satu dependency, yaitu MahasiswaService, dan tidak lagi memuat validasi maupun query SQL. Setiap method hanya memanggil Service, lalu memilih response: menampilkan view atau redirect disertai flash message.

```

class MahasiswaController extends Controller
{
    private MahasiswaService $service;

    public function __construct()
    {
        // Rangkaian dependency: Database -> Repository -> Service -> Controller
        $db = Database::getInstance();

        $this->service = new MahasiswaService(
            new MahasiswaRepository($db),
            new ProdiRepository($db)
        );
    }
}

```

Gambar 10. Constructor MahasiswaController (injeksi dependency)

```

public function store(): void
{
    $result = $this->service->create($_POST);

    if (!$result['success']) {
        $this->backWithErrors($result['errors'], 'mahasiswa/create');
    }

    flash_set('success', 'Data mahasiswa berhasil ditambahkan.');
```

Gambar 11. Method store(): memanggil Service lalu menentukan response

```

public function update(): void
{
    $id      = (int) ($_POST['id'] ?? 0);

```

```

$result = $this->service->update($id, $_POST);

if (!$result['success']) {
    if (!empty($result['not_found'])) {
        flash_set('error', $result['errors']['general']);
        $this->redirect('mahasiswa');
    }
    $this->backWithErrors($result['errors'], 'mahasiswa/edit?id=' . $id);
}

flash_set('success', 'Data mahasiswa berhasil diubah.');
```

Gambar 12. Method update()

```

private function backWithErrors(array $errors, string $path): void
{
    $_SESSION['old_mahasiswa'] = $_POST;
    $_SESSION['form_errors'] = $errors;

    // Satu error (mis. "NIM sudah terdaftar.") ditampilkan apa adanya;
    // beberapa error diringkas menjadi satu pesan "gagal disimpan".
    if (count($errors) === 1) {
        $message = reset($errors);
    } else {
        $message = 'Data gagal disimpan. Periksa kembali isian form.';
    }
    flash_set('error', $message);

    $this->redirect($path);
}
```

Gambar 13. Method backWithErrors(): menyimpan input lama, error per field, dan flash error

Tanggung jawab	Sebelum (Acara 10)	Sesudah (Acara 13)
Validasi data	Controller (lewat exception setter)	MahasiswaService::validate()
Cek NIM duplikat	Tidak ada	Service + Repository::existsByNim()
Cek prodi tersedia	Tidak ada	Service + ProdiRepository::exists()
Query database	Repository (dipanggil Controller)	Repository (dipanggil Service)
Pesan ke pengguna	Hanya pesan sukses	Flash message sukses dan error

Tabel 2. Perbandingan pembagian tanggung jawab

6. Flash Message dengan Session

Fungsi flash_set() dan flash_get() pada config/app.php (dari Acara 6) dipakai ulang. flash_set() menyimpan tipe dan isi pesan ke \$_SESSION['flash'], sedangkan flash_get() mengambil pesan sekaligus menghapusnya dari session (langkah 7 dan 9).

```

function flash_set(string $type, string $message): void
{
    $_SESSION['flash'] = ['type' => $type, 'message' => $message];
}

function flash_get

function flash_get(): ?array
{
    if (empty($_SESSION['flash'])) {
        return null;
    }
    $flash = $_SESSION['flash'];
    unset($_SESSION['flash']);
    return $flash;
}

```

Gambar 14. *flash_set()* dan *flash_get()* pada *config/app.php*

Agar pesan sukses (hijau) dan error (merah) tampil di semua halaman mahasiswa tanpa menulis ulang, dibuat partial *app/Views/partials/flash.php* yang dipanggil pada halaman daftar, tambah, dan ubah (langkah 8). Pesan sukses memudar otomatis lewat JavaScript, sedangkan pesan error tetap tampil sampai halaman berganti. Selain itu, pesan kesalahan per kolom ditampilkan tepat di bawah isian yang salah, dan isian lama dipertahankan sehingga pengguna tidak mengetik ulang.

```

<?php
// app/Views/partials/flash.php
// Menampilkan flash message sesuai tipenya (success / error) dari variabel
// $flash yang dikirim Controller. Pesan sudah dihapus dari session oleh
// flash_get(), jadi hanya tampil satu kali.
if (!empty($flash)) :
    $isError = ($flash['type'] ?? 'success') === 'error';
?>
<div class="<?= $isError ? 'form-alert-error' : 'form-alert-success' ?>"<?=
$isError ? ' ' : ' data-flash' ?>>
    <i class="bi <?= $isError ? 'bi-exclamation-triangle-fill' : 'bi-check-
circle-fill' ?> me-1"></i>
    <?= htmlspecialchars($flash['message']) ?>
</div>
<?php endif; ?>

```

Gambar 15. *Partial flash.php*

Kondisi	Pesan flash	Tipe
Tambah data berhasil	Data mahasiswa berhasil ditambahkan.	success
Ubah data berhasil	Data mahasiswa berhasil diubah.	success
NIM sudah dipakai	NIM sudah terdaftar.	error
Beberapa isian salah / gagal simpan	Data gagal disimpan. Periksa kembali isian form.	error
Kesalahan database	Data gagal disimpan karena terjadi kesalahan pada database.	error
Hapus data berhasil	Data mahasiswa berhasil dihapus.	success

Tabel 3. *Daftar flash message*

7. Pengujian Melalui Browser

Pengujian (langkah 10) dilakukan melalui browser pada <http://localhost/si-akademik/public/> dengan skenario berikut.

No	Skenario	Hasil yang diharapkan
1	Tambah mahasiswa dengan data valid (NIM baru)	Kembali ke daftar, pesan hijau "Data mahasiswa berhasil ditambahkan."
2	Tambah dengan NIM yang sudah ada	Kembali ke form, pesan merah "NIM sudah terdaftar.", isian tetap terisi
3	Tambah dengan NIM berisi huruf	Pesan "NIM harus berupa angka." di bawah kolom NIM
4	Tambah dengan beberapa isian kosong (atribut required dilewati lewat Inspect Element)	Pesan merah "Data gagal disimpan. Periksa kembali isian form." disertai pesan per kolom
5	Ubah nama mahasiswa tanpa mengganti NIM	Pesan hijau "Data mahasiswa berhasil diubah." (NIM sendiri tidak dianggap duplikat)
6	Ubah NIM menjadi NIM mahasiswa lain	Pesan merah "NIM sudah terdaftar."
7	Refresh halaman setelah pesan tampil	Pesan hilang karena flash hanya tampil sekali

Tabel 4. Skenario pengujian

Tambah Mahasiswa

Isi form berikut untuk menambah data mahasiswa baru

NIM

2401004

Nama Lengkap

Feri

Program Studi

Sistem Informasi

Angkatan

2026

Status

Aktif

 Simpan

 Batal

SI Akademik Polije

Dashboard

Data Mahasiswa

Program Studi

Mata Kuliah

Data Dosen

Logout

Data Mahasiswa

Selamat datang, admin

Data mahasiswa berhasil ditambahkan.

Daftar Mahasiswa

5 data

Cari nama atau NIM...

+ Tambah Mahasiswa

NIM	NAMA	PRODI	ANGKATAN	STATUS	AKSI
2401001	Budi Santoso	Teknik Informatika	2024	Aktif	Detail
2401002	Ani Wijaya	Teknik Komputer	2025	Aktif	Detail
2401003	Asti	Teknik Informatika	2026	Aktif	Detail
2401004	Feri	Sistem Informasi	2026	Aktif	Detail
2402001	Citra Lestari	Sistem Informasi	2026	Aktif	Detail

Gambar 16. Flash message berhasil ditambahkan

SI Akademik Polije

Dashboard

Data Mahasiswa

Program Studi

Mata Kuliah

Data Dosen

Logout

Tambah Mahasiswa

Selamat datang, admin

Isi form berikut untuk menambah data mahasiswa baru

NIM sudah terdaftar.

NIM

2401004

NIM sudah terdaftar.

Nama Lengkap

Farel

Program Studi

Teknik Informatika

Angkatan

2026

Status

Aktif

Gambar 17. Flash message NIM sudah terdaftar

Tambah Mahasiswa

Selamat datang, admin

Isi form berikut untuk menambah data mahasiswa baru

⚠ Data gagal disimpan. Periksa kembali isian form.

NIM
abc
NIM harus berupa angka.

Nama Lengkap

Nama wajib diisi.

Program Studi
Teknik Informatika

Angkatan
2026

Status
Aktif

Gambar 18. Flash message data gagal disimpan beserta pesan per kolom

Data Mahasiswa

Selamat datang, admin

Daftar mahasiswa yang terdaftar

✓ Data mahasiswa berhasil diubah.

Daftar Mahasiswa (5 data)

Cari nama atau NIM...

+ Tambah Mahasiswa

NIM	NAMA	PRODI	ANGKATAN	STATUS	AKSI
2401001	Budi Santoso	Teknik Informatika	2024	Cuti	Detail Edit Delete
2401002	Ani Wijaya	Teknik Komputer	2025	Aktif	Detail Edit Delete
2401003	Asti	Teknik Informatika	2026	Aktif	Detail Edit Delete
2401004	Feri	Sistem Informasi	2026	Aktif	Detail Edit Delete
2402001	Citra Lestari	Sistem Informasi	2026	Aktif	Detail Edit Delete

Gambar 19. Flash message berhasil diubah (Budi Santoso yang awalnya aktif menjadi cuti)

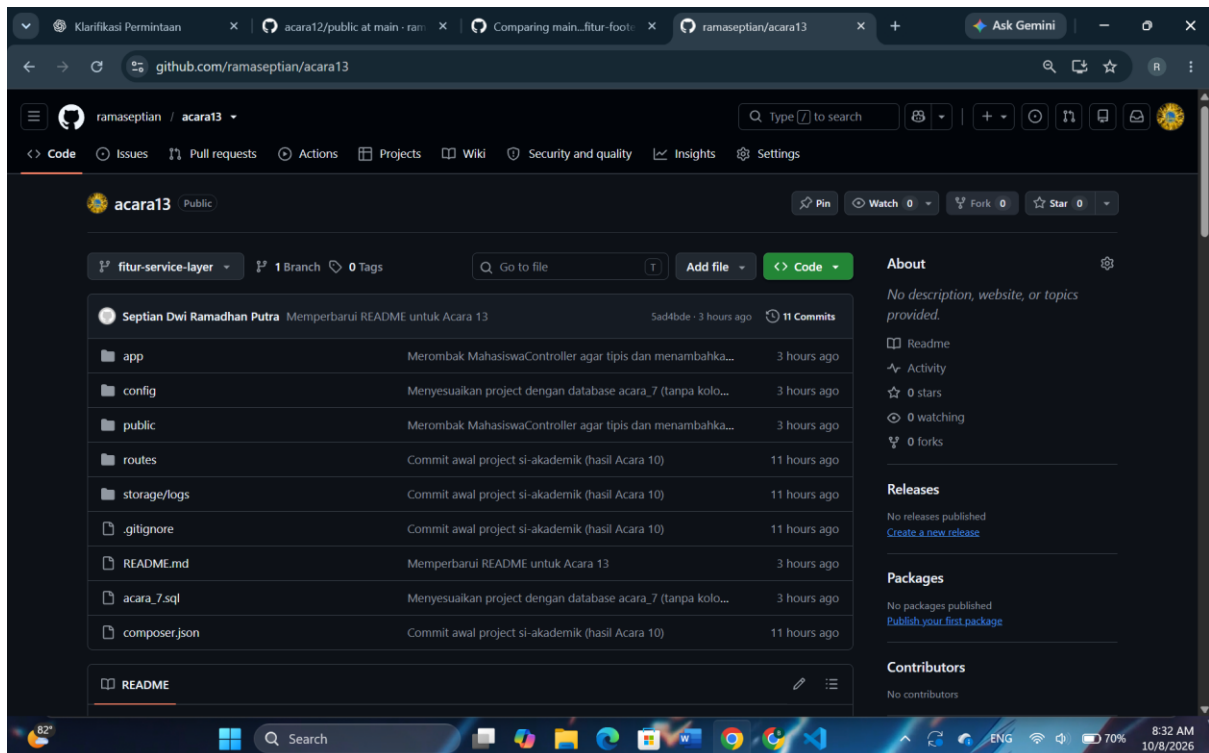
8. Merge ke main dan Log Aktivitas Git

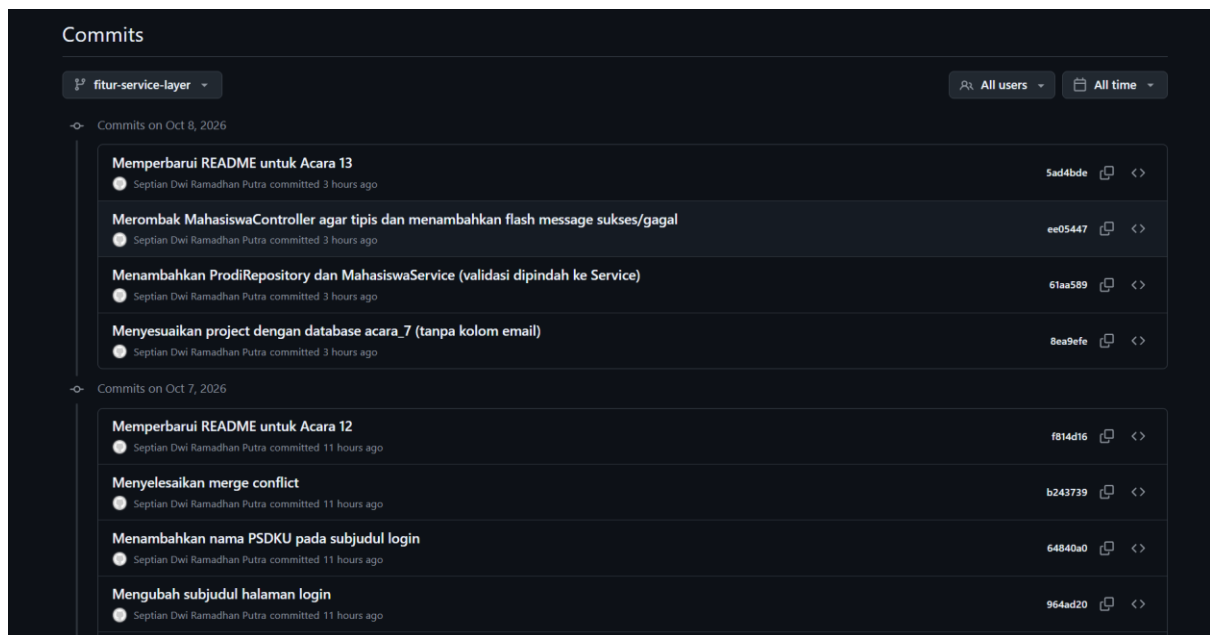
Setelah fitur selesai, branch fitur-service-layer digabungkan ke main menggunakan git merge dengan opsi --no-ff agar riwayat pengerjaan Acara 13 tetap terlihat sebagai satu cabang. Riwayat commit akhir:

```
$ git checkout main
Switched to branch 'main'
$ git merge --no-ff fitur-service-layer
Merge made by the 'ort' strategy.
```

```
$ git log --oneline --graph --decorate -12
* 803b25e (HEAD -> main) Memperbaiki nama database pada README
* a232233 Merge branch 'fitur-service-layer' (Acara 13)
|\
| * 5ad4bde (fitur-service-layer) Memperbarui README untuk Acara 13
| * ee05447 Merombak MahasiswaController agar tipis dan menambahkan flash message
sukses/gagal
| * 61aa589 Menambahkan ProdiRepository dan MahasiswaService (validasi dipindah ke
Service)
| * 8ea9efe Menyesuaikan project dengan database acara_7 (tanpa kolom email)
|/
* f814d16 Memperbarui README untuk Acara 12
* b243739 Menyelesaikan merge conflict
|\
| * 964ad20 (fitur-judul-login) Mengubah subjudul halaman login
* | 64840a0 Menambahkan nama PSDKU pada subjudul login
|/
* 4f41dfa (fitur-login) Menambahkan validasi input kosong dan mempertahankan username
saat login gagal
* 5f0800b Menambahkan tombol lihat/sembunyikan password pada halaman login
```

Gambar 20. Merge fitur-service-layer ke main dan riwayat commit





Gambar 21. Bukti push ke GitHub

9. Ringkasan Output Praktikum

Output yang diminta	Hasil	Bukti
MahasiswaService.php	app/Services/MahasiswaService.php	Gambar 8, 9
Controller yang lebih sederhana	MahasiswaController hanya memanggil Service	Gambar 10 - 13
Validasi berada di Service	MahasiswaService::validate()	Gambar 9
Proses database berada di Repository	MahasiswaRepository dan ProdiRepository	Gambar 6, 7
Flash message setelah create/update	Partial flash.php + <code>\$_SESSION['flash']</code>	Gambar 14 - 19

10. Analisis dan Pembahasan

Pemisahan tiga lapisan membuat setiap class memiliki satu alasan untuk berubah. Jika aturan NIM berubah, hanya MahasiswaService yang diubah; jika struktur tabel berubah, hanya Repository; jika tampilan pesan berubah, hanya Controller dan View. Pada Acara 10 validasi masih bergantung pada exception dari setter sehingga hanya satu kesalahan yang terlihat dalam satu waktu. Dengan validate() yang mengumpulkan semua kesalahan ke dalam array, pengguna dapat melihat seluruh isian yang bermasalah sekaligus.

Pada proses ubah, pengecekan NIM unik harus mengecualikan data yang sedang diubah. Tanpa parameter \$exceptId, menyimpan perubahan tanpa mengganti NIM akan selalu ditolak dengan pesan "NIM sudah terdaftar" karena NIM tersebut milik data itu sendiri. Pengecekan existsByNim() di Service saja belum cukup menjamin keunikan jika dua request datang

bersamaan, sehingga constraint UNIQUE pada database tetap menjadi pengaman terakhir dan kesalahan 1062 diterjemahkan menjadi pesan yang ramah.

Kasus tidak sinkronnya kolom email antara kode dan database acara_7 menunjukkan bahwa kode dan skema database harus selalu diperiksa bersama. Penghapusan email dilakukan konsisten pada seluruh lapisan, dari Entity, Repository, Controller, sampai View. Flash message memakai pola Post/Redirect/Get: setelah POST diproses, pengguna dialihkan ke halaman GET dan pesan diambil dari session lalu dihapus, sehingga me-refresh halaman tidak mengirim ulang form maupun menampilkan pesan yang sama.

KESIMPULAN

1. MahasiswaService berhasil dibuat dengan MahasiswaRepository dan ProdiRepository yang diinjeksikan lewat constructor.
2. Validasi data mahasiswa (NIM wajib, angka, unik; nama; prodi tersedia; angkatan; status) berada di Service, dan proses database berada di Repository, sehingga Controller menjadi lebih sederhana.
3. Method create() dan update() mengembalikan hasil berupa array sukses atau kumpulan error, sehingga alur Controller menjadi jelas dan mudah dibaca.
4. Flash Message berbasis session menampilkan pesan berhasil ditambahkan, berhasil diubah, gagal disimpan, dan NIM sudah terdaftar, kemudian dihapus setelah sekali tampil.
5. Project berhasil disamakan dengan database acara_7 dengan menghapus kolom email dari seluruh lapisan, dan pekerjaan dicatat melalui branch fitur-service-layer yang di-merge ke main.

DAFTAR PUSTAKA

Buku Kerja Praktik Mahasiswa (BKPM) Workshop Sistem Informasi Web Server (TIF330805). Program Studi D4 Teknik Informatika PSDKU Nganjuk, Politeknik Negeri Jember.